

microgpt.py — An Annotated Guide

Training a GPT from Scratch in Pure Python

@karpathy (original)

@nealrichter + Amazon Kiro (annotations)

June 2026

Abstract

This document walks through `microgpt.py`, a minimal implementation of GPT pretraining and inference in pure Python with zero dependencies. Each section of code is presented alongside the mathematical foundations and references to the original papers. The goal is to make the complete algorithm—tokenization, autograd, transformer forward pass, Adam optimization, and autoregressive sampling—accessible to anyone with basic Python and linear algebra knowledge.

Source: <https://github.com/nealrichter/research/blob/main/microgpt/microgpt.py>

Contents

1	Overview	2
2	Data and Tokenization	2
3	Autograd Engine	3
3.1	Forward Operations	3
3.2	Backward Pass	4
4	Model Architecture	4
4.1	Parameter Initialization	4
4.2	Building Blocks	5
4.2.1	Linear Layer	5
4.2.2	Softmax	5
4.2.3	RMSNorm	5
4.3	Transformer Forward Pass	5
4.3.1	Multi-Head Attention	5
4.3.2	MLP Block	6
4.3.3	Output Head	6
5	Training	7
5.1	Loss Function	7
5.2	Adam Optimizer	7
6	Inference	7
7	Model Serialization	8
8	Usage	8

1 Overview

`microgpt.py` implements the full GPT training pipeline in ~ 200 lines:

1. **Data:** Load a corpus of names, tokenize characters to integers.
2. **Autograd:** A scalar-valued automatic differentiation engine.
3. **Model:** A GPT-2-style transformer (attention + MLP).
4. **Training:** Cross-entropy loss, backpropagation, Adam optimizer.
5. **Inference:** Temperature-controlled autoregressive sampling.

Parameter	Value
Layers	1
Embedding dimension	16
Attention heads	4
Context length	16
MLP hidden dimension	64
Total parameters	~ 4500

Table 1: Model architecture

2 Data and Tokenization

The model trains on a character-level corpus of names. Each unique character becomes a token ID, plus a special Beginning-of-Sequence (BOS) token.

```
14 import os, sys, math, random, json
15 random.seed(42)

46 if not os.path.exists('input.txt'):
47     import urllib.request
48     names_url = 'https://raw.githubusercontent.com/karpathy/
49               makemore/988aa59/names.txt'
49     urllib.request.urlretrieve(names_url, 'input.txt')
50 docs = [line.strip() for line in open('input.txt') if line.strip()]
51 random.shuffle(docs)
52
53 uchars = sorted(set(''.join(docs)))
54 BOS = len(uchars)
55 vocab_size = len(uchars) + 1
```

The tokenizer maps each character c to its index in the sorted unique character set:

$$\text{encode}(c) = \text{index}(c, \text{uchars}), \quad \text{decode}(i) = \text{uchars}[i]$$

Documents are wrapped with BOS tokens: $[\text{BOS}, t_1, t_2, \dots, t_n, \text{BOS}]$.

3 Autograd Engine

The `Value` class implements scalar-valued reverse-mode automatic differentiation. Each node stores its value, gradient, children, and local gradients (partial derivatives with respect to its children).

```
60 class Value:
61     __slots__ = ('data', 'grad', '_children', '_local_grads')
62
63     def __init__(self, data, children=(), local_grads=()):
64         self.data = data
65         self.grad = 0
66         self._children = children
67         self._local_grads = local_grads
```

3.1 Forward Operations

Each arithmetic operation creates a new node and records the local gradients for the chain rule:

```
69     def __add__(self, other):
70         other = other if isinstance(other, Value) else Value(other)
71         return Value(self.data + other.data, (self, other), (1, 1))
72
73     def __mul__(self, other):
74         other = other if isinstance(other, Value) else Value(other)
75         return Value(self.data * other.data, (self, other), (other.data, self.data))
76
77     def __pow__(self, other):
78         return Value(self.data**other, (self,), (other * self.data
79             **(other-1),))
79
80     def log(self):
81         return Value(math.log(self.data), (self,), (1/self.data,))
82
83     def exp(self):
84         return Value(math.exp(self.data), (self,), (math.exp(self.data),))
85
86     def relu(self):
87         return Value(max(0, self.data), (self,), (float(self.data > 0),))
```

The local gradients follow directly from calculus:

$$\frac{\partial}{\partial a}(a + b) = 1 \quad (1)$$

$$\frac{\partial}{\partial a}(a \cdot b) = b \quad (2)$$

$$\frac{\partial}{\partial x}(x^n) = n \cdot x^{n-1} \quad (3)$$

$$\frac{\partial}{\partial x} \ln(x) = \frac{1}{x} \quad (4)$$

$$\frac{\partial}{\partial x} e^x = e^x \quad (5)$$

$$\frac{\partial}{\partial x} \text{ReLU}(x) = \mathbf{1}[x > 0] \quad (6)$$

3.2 Backward Pass

Backpropagation computes gradients via topological sort and the chain rule [4]:

```
93     def backward(self):
94         topo = []
95         visited = set()
96         def build_topo(v):
97             if v not in visited:
98                 visited.add(v)
99                 for child in v._children:
100                     build_topo(child)
101             topo.append(v)
102         build_topo(self)
103         self.grad = 1
104         for v in reversed(topo):
105             for child, local_grad in zip(v._children, v._local_grads):
106                 child.grad += local_grad * v.grad
```

For any node v with children $c_1, \dots, c_k$, the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial c_i} += \frac{\partial v}{\partial c_i} \cdot \frac{\partial \mathcal{L}}{\partial v}$$

4 Model Architecture

The architecture follows GPT-2 [2] with minor simplifications: RMSNorm [5] replaces LayerNorm, biases are omitted, and ReLU replaces GeLU.

4.1 Parameter Initialization

Weights are initialized from $\mathcal{N}(0, 0.08^2)$:

```
109 n_layer = 1
110 n_embd = 16
111 block_size = 16
112 n_head = 4
113 head_dim = n_embd // n_head
114
115 matrix = lambda nout, nin, std=0.08: \
116     [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in
117      range(nout)]
118
119 state_dict = {
120     'wte': matrix(vocab_size, n_embd),
121     'wpe': matrix(block_size, n_embd),
122     'lm_head': matrix(vocab_size, n_embd),
123 }
124 for i in range(n_layer):
125     state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
126     state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
127     state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
128     state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
129     state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
130     state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
```

4.2 Building Blocks

4.2.1 Linear Layer

A matrix-vector product $y = Wx$:

```
128 def linear(x, w):  
129     return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]
```

4.2.2 Softmax

The numerically stable softmax function:

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}}$$

```
131 def softmax(logits):  
132     max_val = max(val.data for val in logits)  
133     exps = [(val - max_val).exp() for val in logits]  
134     total = sum(exps)  
135     return [e / total for e in exps]
```

4.2.3 RMSNorm

Root Mean Square Layer Normalization [5] normalizes by the RMS of the input, without centering:

$$\text{RMSNorm}(x_i) = \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}}$$

```
137 def rmsnorm(x):  
138     ms = sum(xi * xi for xi in x) / len(x)  
139     scale = (ms + 1e-5) ** -0.5  
140     return [xi * scale for xi in x]
```

4.3 Transformer Forward Pass

The GPT model processes one token at a time, maintaining a KV-cache for efficient autoregressive generation:

```
142 def gpt(token_id, pos_id, keys, values):  
143     tok_emb = state_dict['wte'][token_id]  
144     pos_emb = state_dict['wpe'][pos_id]  
145     x = [t + p for t, p in zip(tok_emb, pos_emb)]  
146     x = rmsnorm(x)
```

The input representation combines token and positional embeddings [1]:

$$x = \text{RMSNorm}(W_{\text{te}}[t] + W_{\text{pe}}[p])$$

4.3.1 Multi-Head Attention

Scaled dot-product attention [1] with causal masking (implemented implicitly via the KV-cache—only past keys/values are stored):

```

150     x_residual = x
151     x = rmsnorm(x)
152     q = linear(x, state_dict[f'layer{li}.attn_wq'])
153     k = linear(x, state_dict[f'layer{li}.attn_wk'])
154     v = linear(x, state_dict[f'layer{li}.attn_wv'])
155     keys[li].append(k)
156     values[li].append(v)
157     x_attn = []
158     for h in range(n_head):
159         hs = h * head_dim
160         q_h = q[hs:hs+head_dim]
161         k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
162         v_h = [vi[hs:hs+head_dim] for vi in values[li]]
163         attn_logits = [sum(q_h[j] * k_h[t][j]
164                             for j in range(head_dim)) / head_dim**0.5
165                         for t in range(len(k_h))]
166         attn_weights = softmax(attn_logits)
167         head_out = [sum(attn_weights[t] * v_h[t][j]
168                         for t in range(len(v_h))) for j in range(head_dim)
169                     ]
170         x_attn.extend(head_out)
171     x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
172     x = [a + b for a, b in zip(x, x_residual)]

```

For each head h , the attention computation is:

$$Q = W_Q \cdot x, \quad K = W_K \cdot x, \quad V = W_V \cdot x \quad (7)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (8)$$

where $d_k = d_{\text{embd}}/n_{\text{heads}} = 4$ is the per-head dimension. The $\sqrt{d_k}$ scaling prevents dot products from growing too large [1].

The output projection W_O recombines the concatenated head outputs, followed by a residual connection [6].

4.3.2 MLP Block

A two-layer feed-forward network with ReLU activation and $4\times$ expansion:

```

172     x_residual = x
173     x = rmsnorm(x)
174     x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
175     x = [xi.relu() for xi in x]
176     x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
177     x = [a + b for a, b in zip(x, x_residual)]

```

$$\text{MLP}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot \text{RMSNorm}(x))$$

The original GPT-2 uses GeLU [7]; ReLU is used here for simplicity.

4.3.3 Output Head

The final linear projection maps from embedding space to vocabulary logits:

```

179     logits = linear(x, state_dict['lm_head'])
180     return logits

```

5 Training

5.1 Loss Function

The training objective is cross-entropy loss over next-token prediction:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \log P(t_{i+1} | t_1, \dots, t_i)$$

```
196     for pos_id in range(n):
197         token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
198         logits = gpt(token_id, pos_id, keys, values)
199         probs = softmax(logits)
200         loss_t = -probs[target_id].log()
201         losses.append(loss_t)
202     loss = (1 / n) * sum(losses)
```

5.2 Adam Optimizer

The Adam optimizer [3] with linear learning rate decay:

```
182     learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
183     m = [0.0] * len(params)
184     v = [0.0] * len(params)
185
186     lr_t = learning_rate * (1 - step / num_steps)
187     for i, p in enumerate(params):
188         m[i] = beta1 * m[i] + (1 - beta1) * p.grad
189         v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
190         m_hat = m[i] / (1 - beta1 ** (step + 1))
191         v_hat = v[i] / (1 - beta2 ** (step + 1))
192         p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
193     p.grad = 0
```

The Adam update rules with bias correction:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (11)$$

$$\theta_t = \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (12)$$

The learning rate decays linearly: $\eta_t = \eta_0 \cdot (1 - t/T)$.

6 Inference

Autoregressive generation with temperature scaling:

```
232     temperature = 0.5
233     token_id = BOS
234     for pos_id in range(block_size):
235         logits = gpt(token_id, pos_id, keys, values)
236         probs = softmax([l / temperature for l in logits])
237         token_id = random.choices(range(vocab_size),
238                                 weights=[p.data for p in probs])[0]
```

```

239     if token_id >= len(uchars):
240         break
241     sample.append(uchars[token_id])

```

Temperature $\tau \in (0, 1]$ controls the sharpness of the distribution:

$$P(t_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

As $\tau \rightarrow 0$, sampling becomes greedy (always picks the most likely token). As $\tau \rightarrow 1$, the original distribution is preserved.

7 Model Serialization

After training, weights are saved as JSON for later inference or fine-tuning:

```

220 with open('model.json', 'w') as f:
221     json.dump({'vocab': uchars,
222               'config': {'n_layer': n_layer, 'n_embd': n_embd,
223                           'block_size': block_size, 'n_head':
224                               n_head},
225               'weights': {k: [[p.data for p in row] for row in
226                               mat]
227                           for k, mat in state_dict.items()}}), f)

```

The `-i` flag loads a saved model and runs inference without retraining:

```

1 python3 microgpt.py -i                    # default: model.json
2 python3 microgpt.py -i model_sft.json    # from SFT model

```

8 Usage

```

1 # Pretrain base model (trains on names, saves model.json)
2 python3 microgpt.py
3
4 # Inference only from saved model (different results each run)
5 python3 microgpt.py -i
6 python3 microgpt.py -i model_sft.json

```

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems 30, 2017. <https://arxiv.org/abs/1706.03762>
- [2] Radford, A., Wu, J., Child, R., et al. *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [3] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. Proceedings of ICLR, 2015. <https://arxiv.org/abs/1412.6980>
- [4] Rumelhart, D.E., Hinton, G.E., and Williams, R.J. *Learning Representations by Back-propagating Errors*. Nature 323, pp. 533–536, 1986.

- [5] Zhang, B. and Sennrich, R. *Root Mean Square Layer Normalization*. Advances in Neural Information Processing Systems 32, 2019. <https://arxiv.org/abs/1910.07467>
- [6] He, K., Zhang, X., Ren, S., and Sun, J. *Deep Residual Learning for Image Recognition*. Proceedings of CVPR, 2016. <https://arxiv.org/abs/1512.03385>
- [7] Hendrycks, D. and Gimpel, K. *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415, 2016. <https://arxiv.org/abs/1606.08415>